

Reinforcement Learning: Model-Free Value Prediction (ESE 3990)

Instructor:
Antonio Loquercio

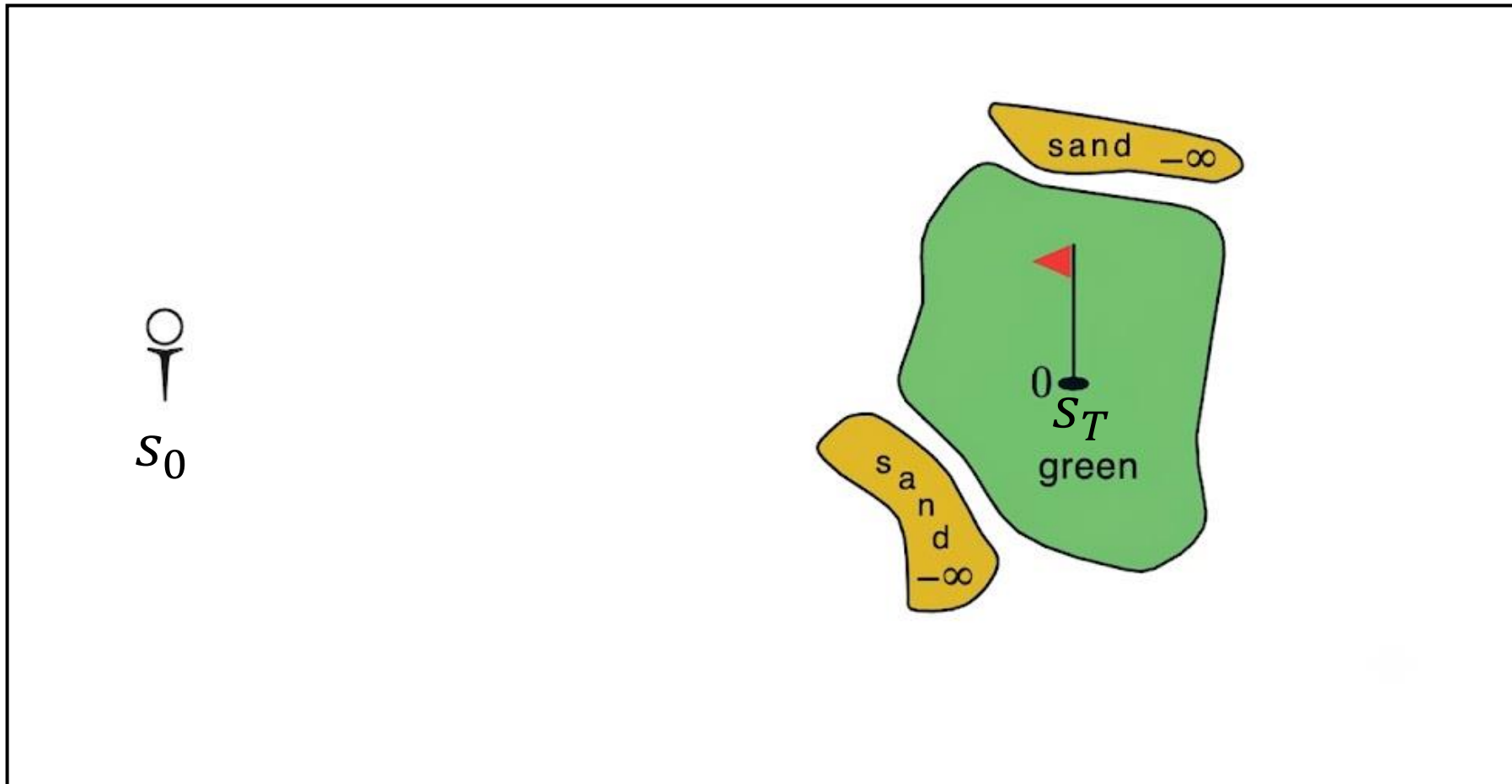
Fall 2026
University of Pennsylvania

What We Will Cover in the Next Classes

- Solve a *known* MDP
 - Planning by Dynamic Programming
- Given π , evaluate the value function of an *unknown* MDP
 - Model-free prediction
- Optimize the value function/policy of an *unknown* MDP
 - Model-free control

Example: Playing Golf

$$s \in S$$



$$a \in A$$



Driver

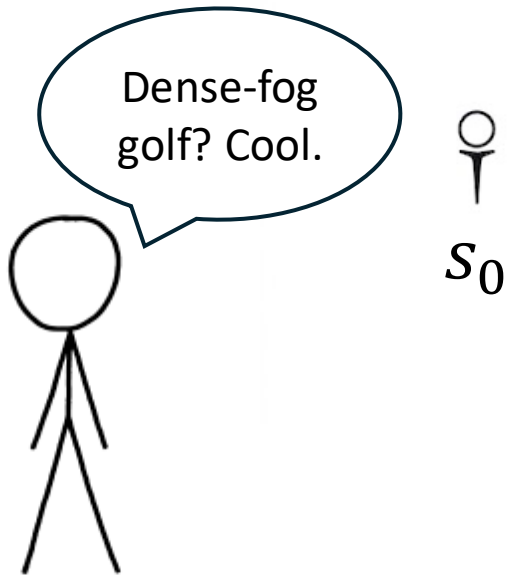


Putt

$$r(s, a) = -1$$

Finding V^π in a novel field with fog

- You only know where you start, but nothing else about the field.
- Assume there is dense fog, and you can't see anything beyond the location of the ball (because it has GPS).
- Can you use the same technique as before?



$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

Finding V^π in a novel field with fog

- What would you do to find V^π ?

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$


 s_0

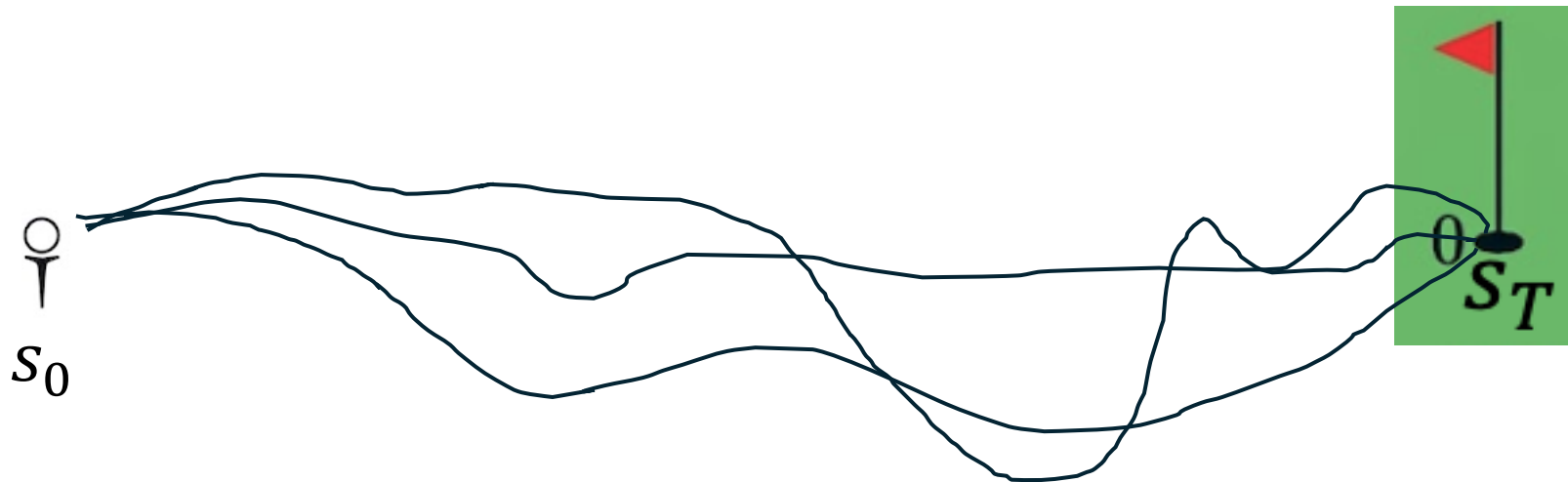
Finding V^π in a novel field with fog

$$r(s, a) = -1$$

- Idea 1: Trial and Error.
 - Act until you hit the hole.
 - Assume that the states you observe are $s_1, s_2, \dots, s_T$.
- What is $\hat{V}^\pi(s_t)$?
- You might want to repeat the procedure a few times if the environment or the policy are stochastic.



$$\pi(s) = \text{Putt}$$



Monte-Carlo Reinforcement Learning



- MC methods learn directly from episodes of experience
- MC is model-free
 - *No prior knowledge* of MDP transitions or rewards
- MC learns from complete episodes.
 - Caveat: can only apply MC to *episodic* MDPs, where all episodes terminate.
- MC uses the simplest possible idea:
 - Value = Mean Return over some trials

Monte-Carlo Policy Evaluation

- Goal: learn V_π from episodes of experience under policy π

$$s_1, a_1, r_1, \dots, s_k \sim \pi$$

- Refreshers:

- The *return* G_t is the total discounted reward: $G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-1} r_T$
- The *value function* is the expected return: $V_\pi(s) = \mathbb{E}[G_t | s_t = s]$

- Monte-Carlo policy evaluation uses *empirical mean return* as an *estimator* of expected return.

- $V_\pi^{MC}(s) = \frac{1}{N} \sum_n G_t^n$, where N is the total number of episodes starting in s.

Algorithm: Every-Visit MC Policy Evaluation

- Goal: evaluate state s in a tabular environment
- Approach:
 - Every time-step t that state s is visited in an episode
 - Increment counter $N(s) \leftarrow N(s) + 1$
 - Increment total return $S(s) \leftarrow S(s) + G_t$
 - Value is estimated by mean return $V(s) = \frac{S(s)}{N(s)}$
- Easy to show that $V(s) \rightarrow V_{\pi}(s)$ as $N(s) \rightarrow \infty$

Incremental Mean

- The mean $\mu_1, \mu_2, \dots$ of a sequence $x_1, x_2, \dots$ can be computed incrementally

$$\mu_k = \frac{1}{k} \sum_{j=1}^k x_j$$

Incremental Monte-Carlo Updates

- *Idea*: Update $V(s)$ incrementally after episodes $s_1, a_1, r_2, \dots, r_T$
- *Implementation*:
 - For each state s_t with return G_t
 - $N(s_t) \leftarrow N(s_t) + 1$
 - $V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)} (G_t - V(s_t))$
- If a problem is *non-stationary* (rules change over time), it is useful to track a running mean, i.e. gradually forget old episodes:
 - $V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t))$

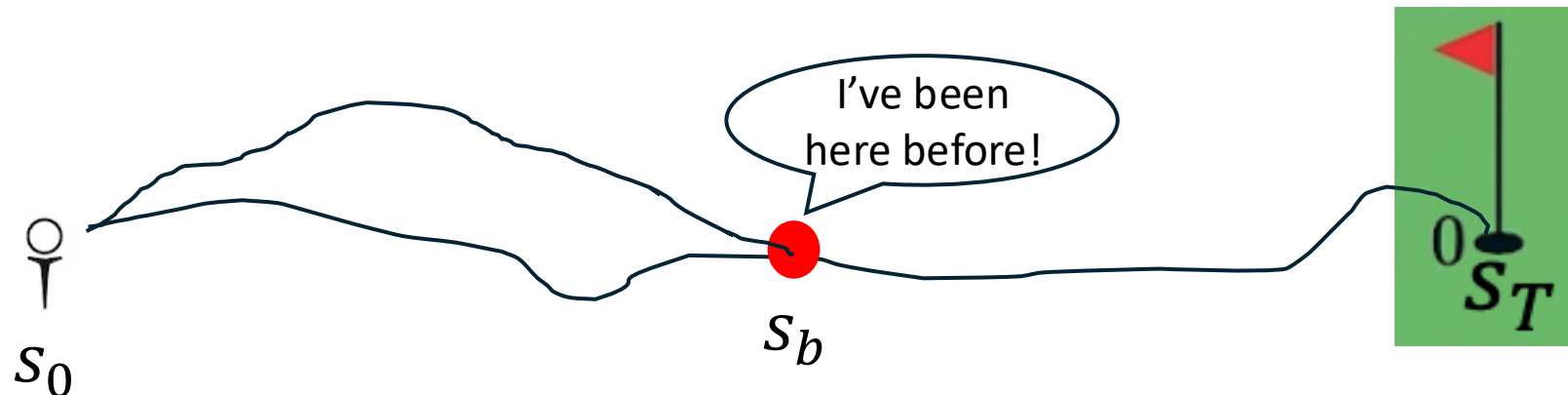
Finding V^π in a novel field with fog

- Idea 2: Reuse prior experience.
- Trial 1: Do the same as MC. Try until you hit.
- In the next trial, I end up in a state s_b visited earlier.
- Can I get an estimate of $\hat{V}^\pi(s_b)$ without terminating the episode?

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



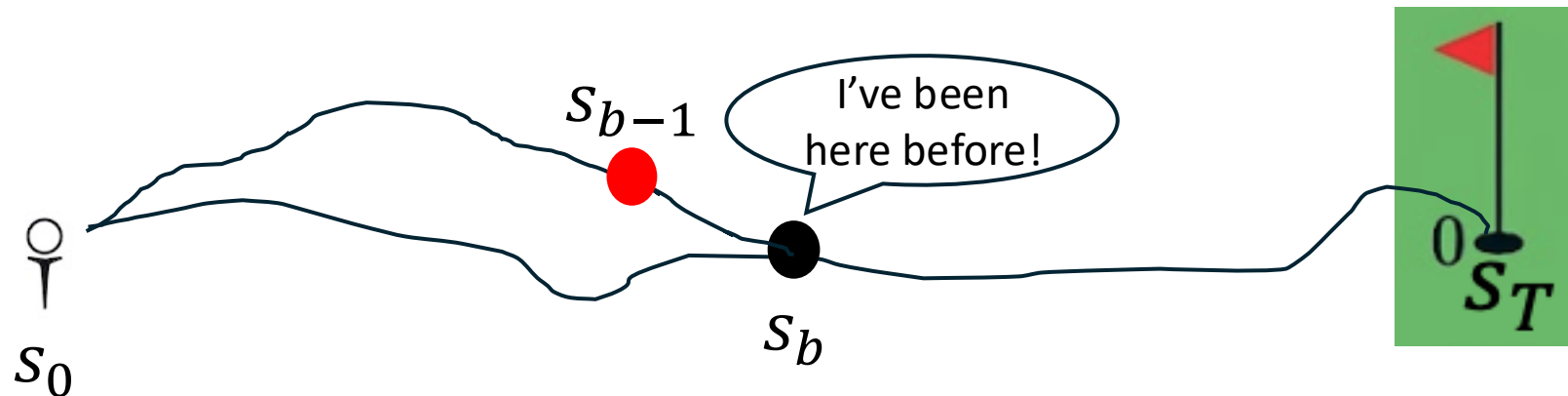
Finding V^π in a novel field with fog

- Idea 2: Reuse prior experience.
- If s_{b-1} is the state immediately before s_b , can we get an estimate of its value before concluding the episode?

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



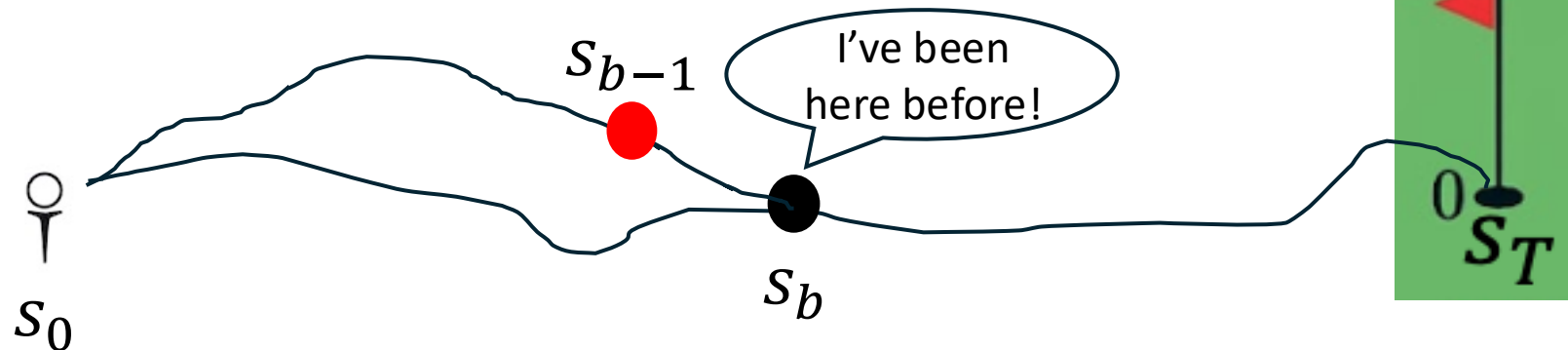
Finding V^π in a novel field with fog

- Yes!
- $\hat{V}^\pi(s_{b-1}) = r_{b-1} + \gamma \hat{V}^\pi(s_b) = -1 + \hat{V}^\pi(s_b)$
- I can do this backwards for all prior states, and get a second estimate of $\hat{V}^\pi(s_0)$ without episode termination!
- This process is called *bootstrapping*.

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



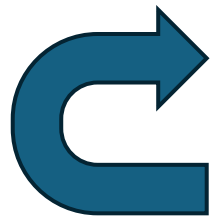
Temporal-Difference (TD) Learning

- TD learns from *incomplete episodes*, by bootstrapping
 - This is the key difference from MC
- *Intuition*: TD updates a guess towards a guess
- TD is model-free:
 - No knowledge of MDP transitions / rewards
 - Learns directly from experience

TD(0): The simplest algorithm

Goal: Find V^π given π

Algorithm:



- Initialize $V(s) = 0 \forall s \in S$
- Rollout π for one or more episodes
- $\forall s_t$ observed by π , update value $V(s_t)$ toward *estimated* return $r_{t+1} + \gamma V(s_{t+1})$
- $V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$

• *Definitions:*

- *TD target:* $r_{t+1} + \gamma V(s_{t+1})$
- *TD error:* $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$

Monte-Carlo vs Temporal-Difference Learning

Incremental every-visit MC:

- Update value $V(s_t)$ toward *actual* return G_t
- $V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$

TD(0), the simplest TD algorithm:

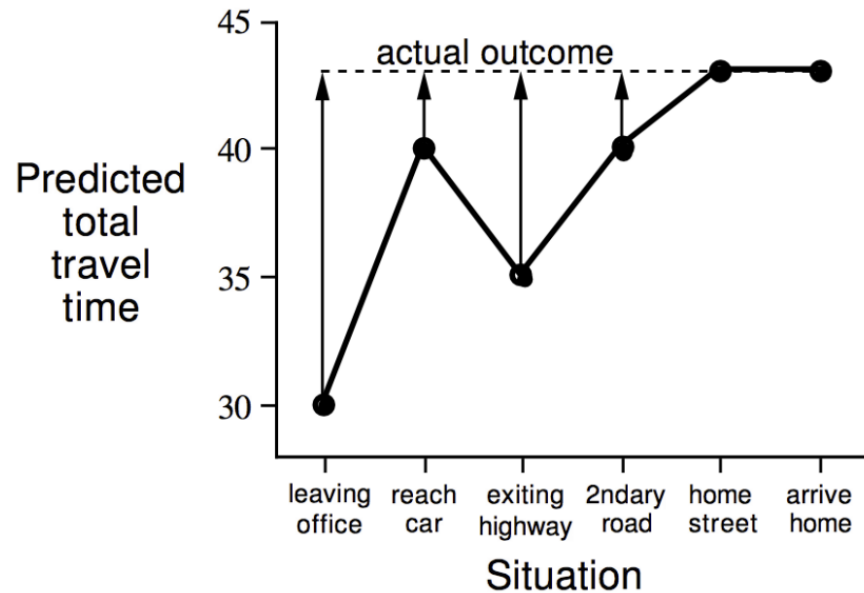
- Update value $V(s_t)$ toward *estimated* return $r_{t+1} + \gamma V(s_{t+1})$
- $V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$

Comparing MC and TD by Driving Home

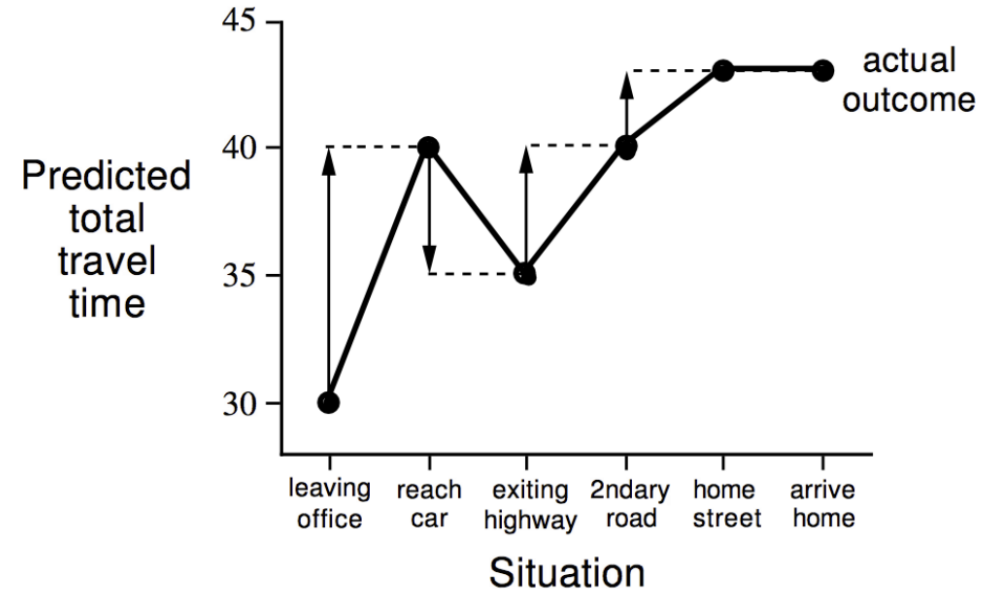
State	Elapsed Time (minutes)	Predicted Time to Go	Predicted Total Time
leaving office	0	30	30
reach car, raining	5	35	40
exit highway	20	15	35
behind truck	30	10	40
home street	40	3	43
arrive home	43	0	43

Example: Driving Home. MC vs TD.

Changes recommended by
Monte Carlo methods ($\alpha=1$)



Changes recommended
by TD methods ($\alpha=1$)

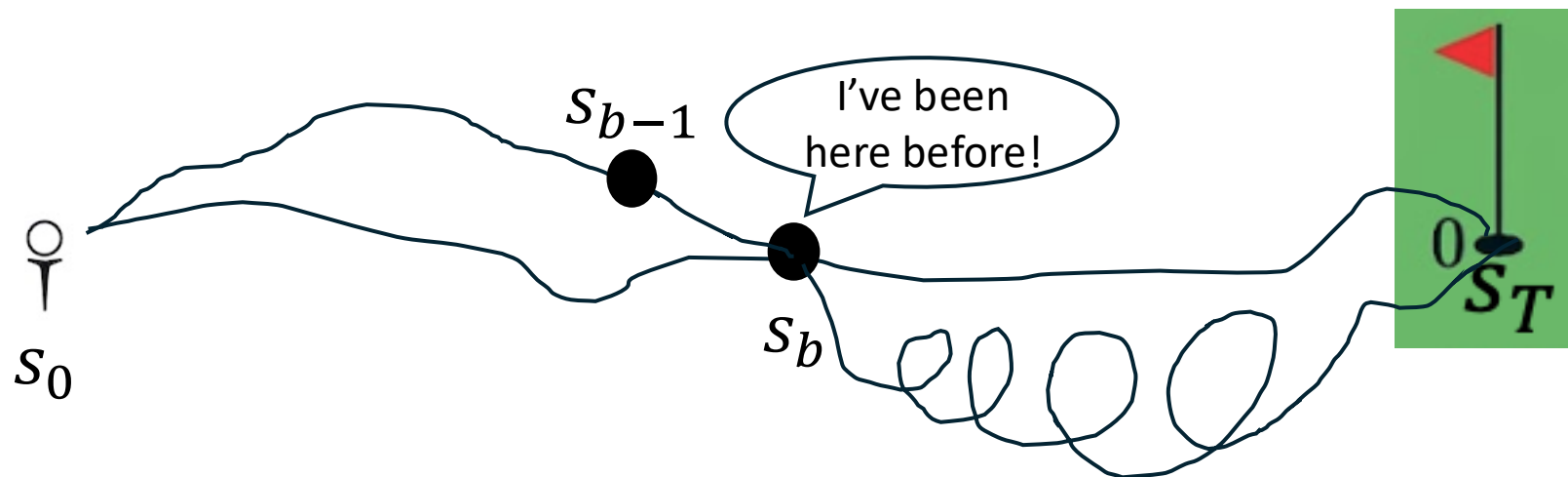


MC vs TD: Online Learning

- TD can learn online *after every step*
 - If you have not seen the state before, you can use a *guess* of the value of that state.
- TD can learn *without* the final outcome
 - TD can learn from *incomplete* sequences
 - Therefore, we can use TD in continuing (non-terminating) environments
- MC only works for episodic (terminating) environments!

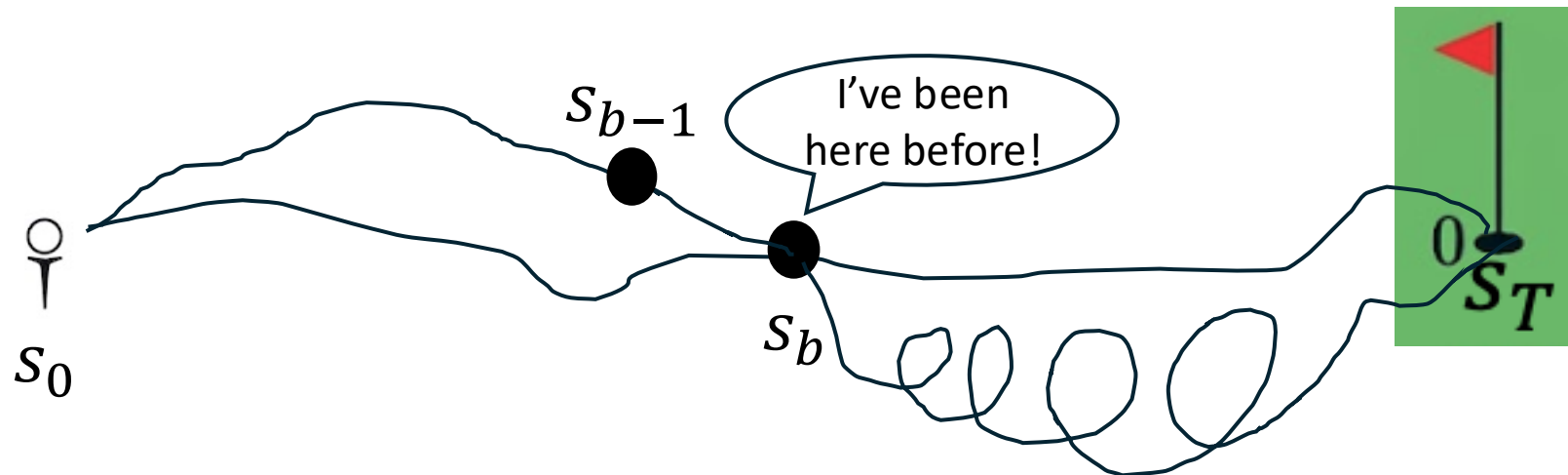
Finding V^π in a novel field with fog

- Assume that I continue the episode after s_b , and end up taking much longer the second time to hit the hole.
- Using online TD(0), is the estimate of $\hat{V}^\pi(s_{b-1})$ affected by what happens after s_b ?
- What about my second estimate of $\hat{V}^\pi(s_0)$?



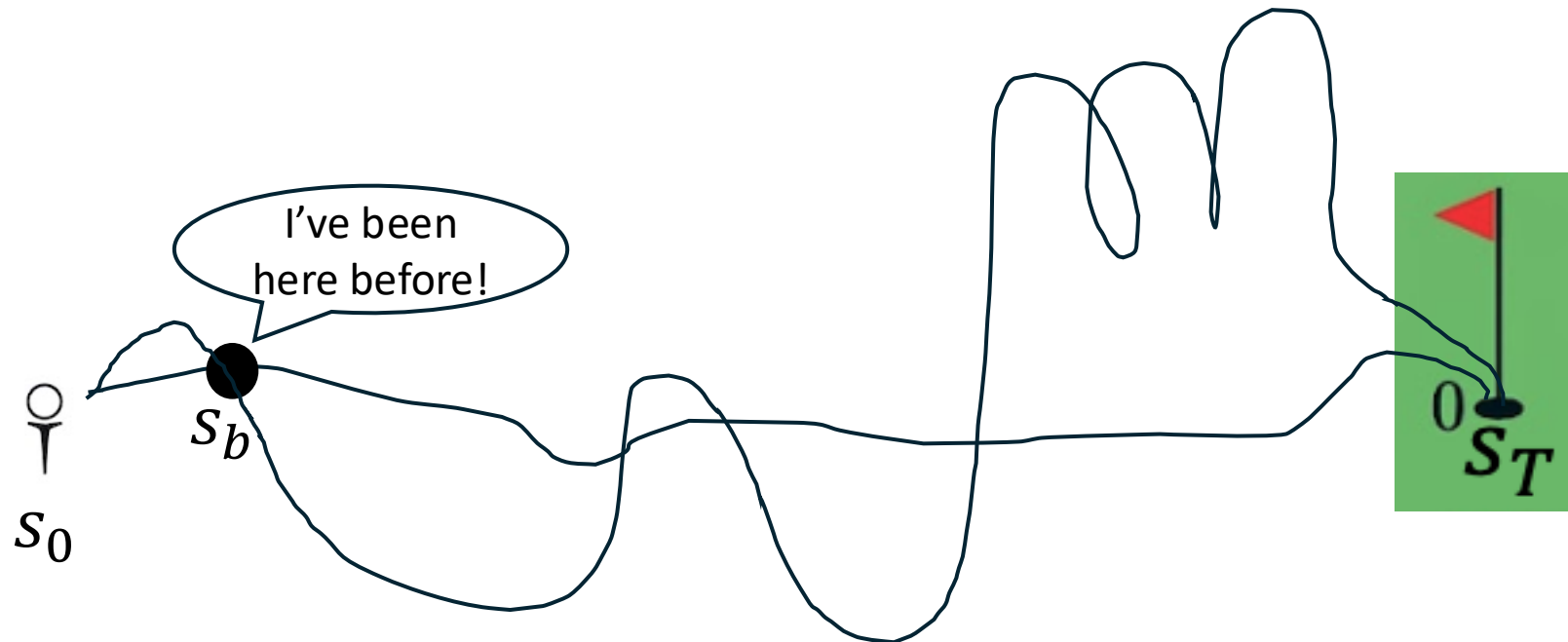
Finding V^π in a novel field with fog

- TD(0) does not care of what happens in the whole episode, but only about what happens in the next step.
- This makes it shortsighted, and likely *to overestimate or underestimate* the true value of a state.
- The TD(0) value estimator is **biased**.



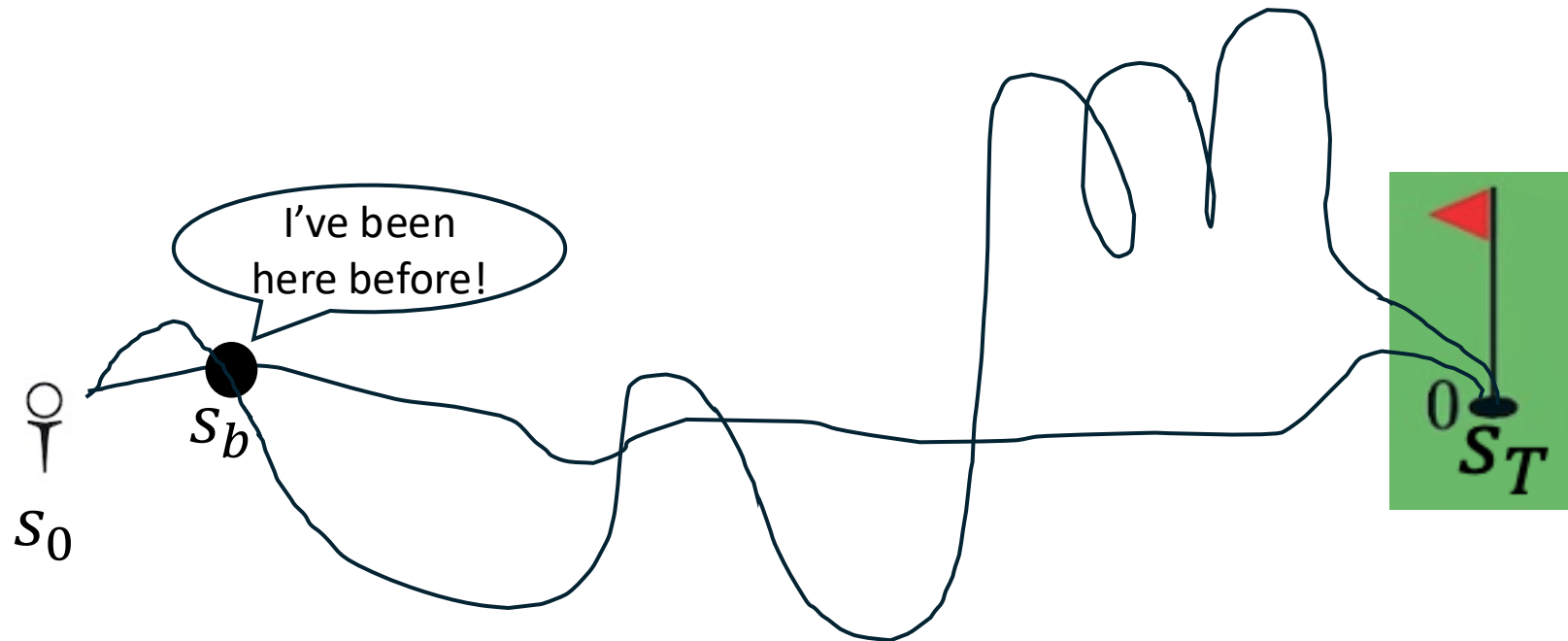
Finding V^π in a novel field with fog

- The TD(0) value estimator is **biased**. But that also comes with advantages!
- Call $\hat{V}_1^\pi(s_0)$, $\hat{V}_2^\pi(s_0)$ the value estimates obtained from two runs.
- For MC, $|\hat{V}_1^\pi(s_0) - \hat{V}_2^\pi(s_0)|$ is large (because the runs are very different).
- For TD(0), is $|\hat{V}_1^\pi(s_0) - \hat{V}_2^\pi(s_0)|$ going to be larger, smaller, or equal?



Finding V^π in a novel field with fog

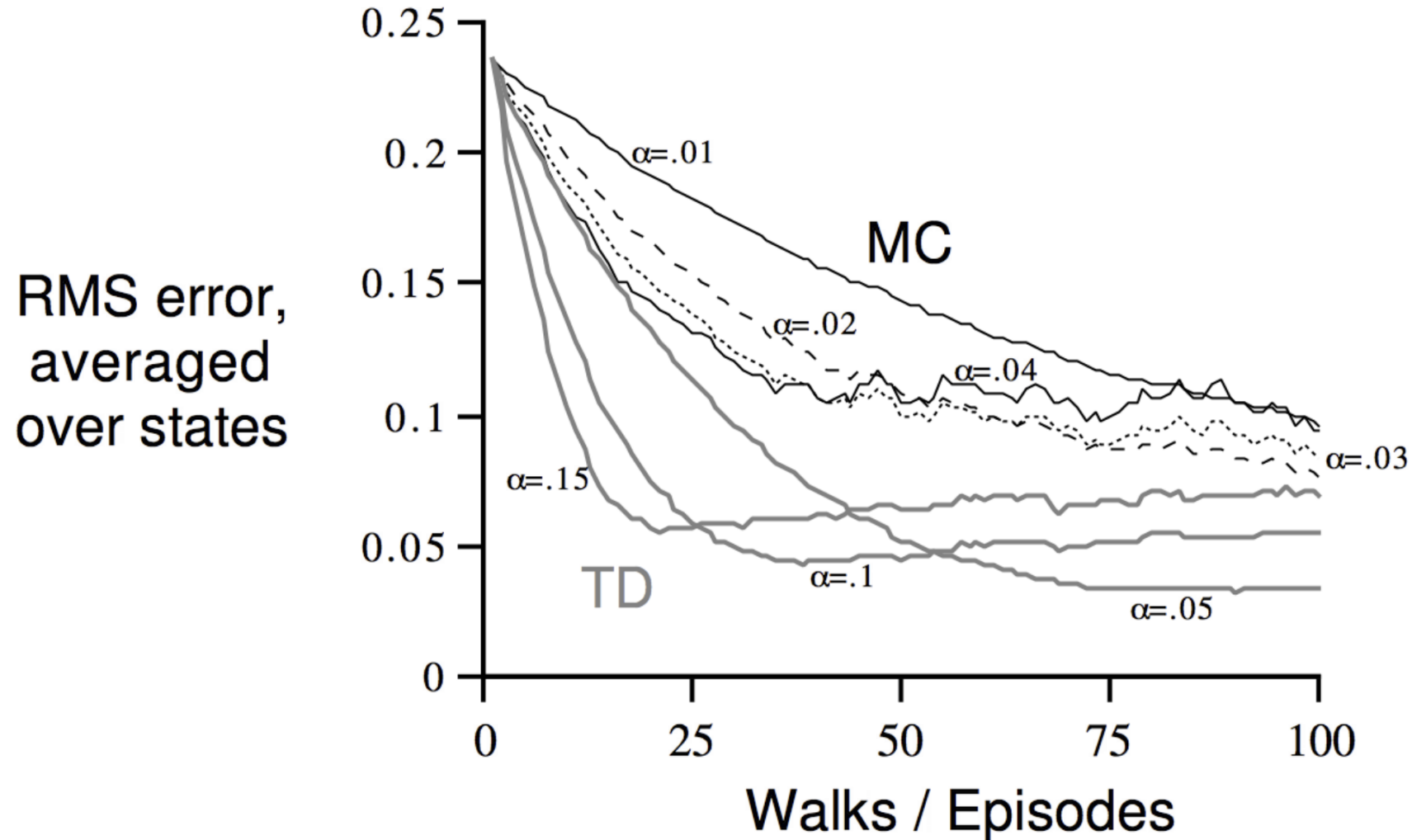
- In general, the difference between estimations of $\hat{V}^\pi(s)$ is smaller for TD(0) than MC.
- This can be very beneficial if you're trying to fit a function to $V^\pi(s)$.
- We say that TD(0) has **lower variance** than MC.



MC vs TD: Bias/Variance Trade-Off

- MC has high variance, zero bias
 - Good convergence properties (even with function approximation)
 - Not very sensitive to value initialization
 - Very simple to understand and use
- TD has low variance, but some bias
 - Usually more sample efficient than MC
 - TD(0) converges to $V^\pi(s)$ (but not always with function approximation)
 - More sensitive to value initialization

Random Walk: MC vs. TD



Online vs Batch MC / TD

- Until now, we have updated our estimate of the value function either at every step (for TD) or at the end of the episode (for MC).
- Both MC and TD converge: $V(s) \rightarrow V_{\pi}(s)$ as experience $\rightarrow \infty$.
- However, after the update, we throw away the sample/episode. **Can we do better?**
- We can be even more efficient by running again updates using *batches* of previous data.

Batch MC / TD

- Assume that we have sampled, up to now, K full episodes:

$$s_1^1, a_1^1, r_2^1, \dots, s_T^1$$

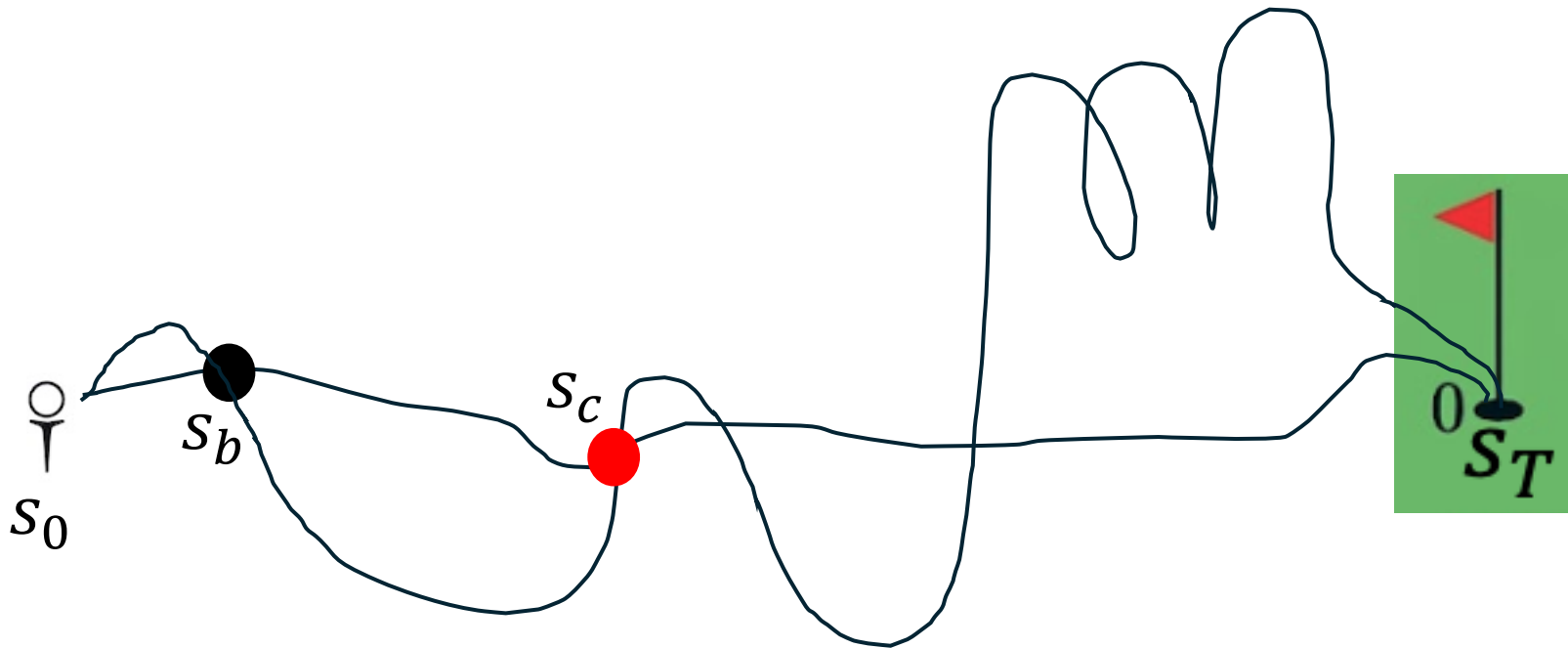
...

$$s_1^K, a_1^K, r_2^K, \dots, s_T^K$$

- *Implementation:* Repeatedly sample episode $k \in [1, K]$, and apply MC or TD to it.

Why is it helpful to revisit old episodes?

- Example: Batch TD(0).
- The experience of the "crazy" trajectory now propagates back to s_b
- *Less bias!*



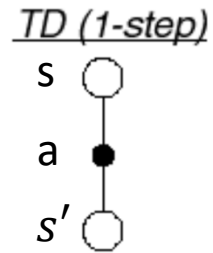
Convergence: Batch MC vs TD

- TD exploits the Markov property
 - Usually more *data efficient* in Markov environments
- MC does not exploit the Markov property
 - Usually more *effective* in non-Markov environments.

The Connection between TD(0) and MC

n-step Prediction

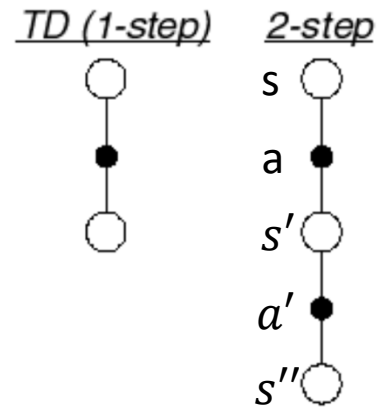
- Up to now. TD learning looks at one step into the future:



$$V(s) = r(s, a) + \gamma V(s')$$

n-step Prediction

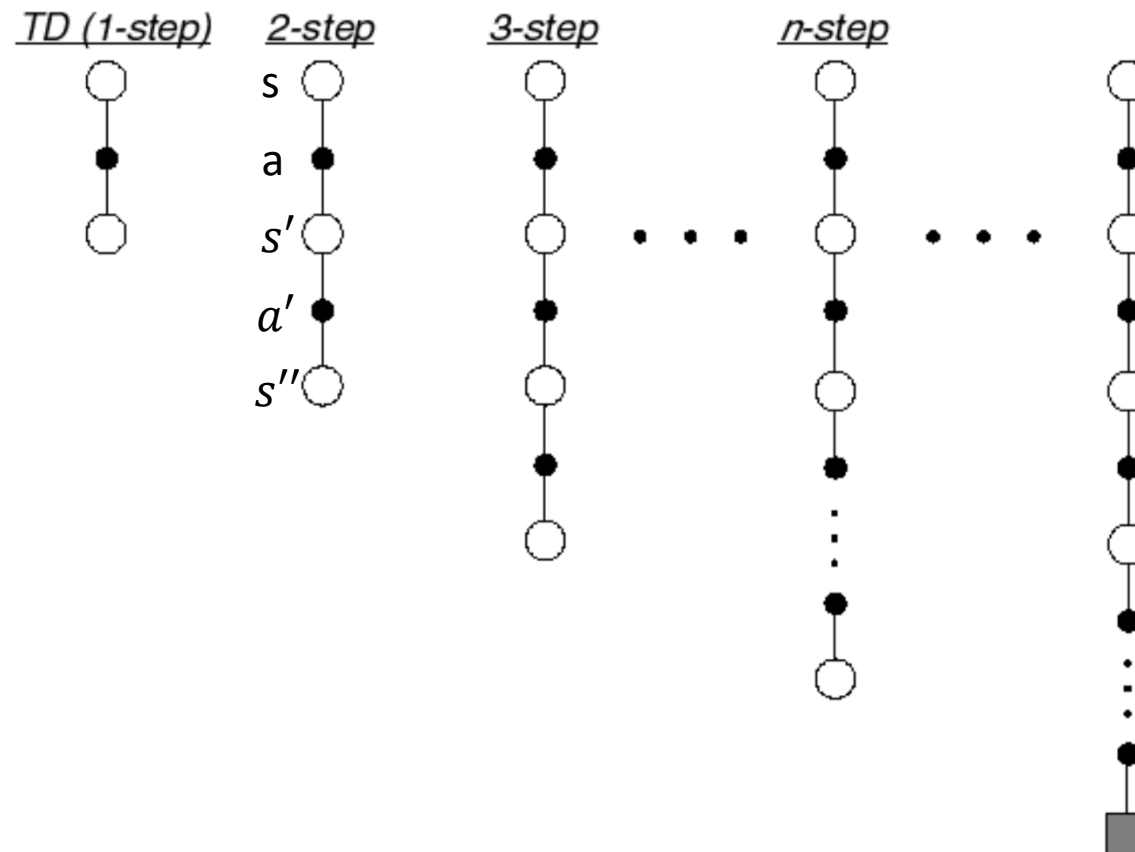
- But why stopping at one step? Why not 2?



$$V(s) = r(s, a) + \gamma r(s', a') + \gamma^2 V(s'')$$

n-step Prediction

- We can use as many steps as the length of the episode.
- How is it called a TD-learning step that uses the entire episode?



N-Step Return

- Consider the following n-step returns for $n = 1, 2, \infty$

$$\begin{array}{ll} n = 1 & (TD) \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \\ n = 2 & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \\ & \vdots \\ n = \infty & (MC) \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T \end{array}$$

- Define the n-step return

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

- N-step temporal-difference learning: Just pick one.

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{(n)} - V(s_t) \right)$$

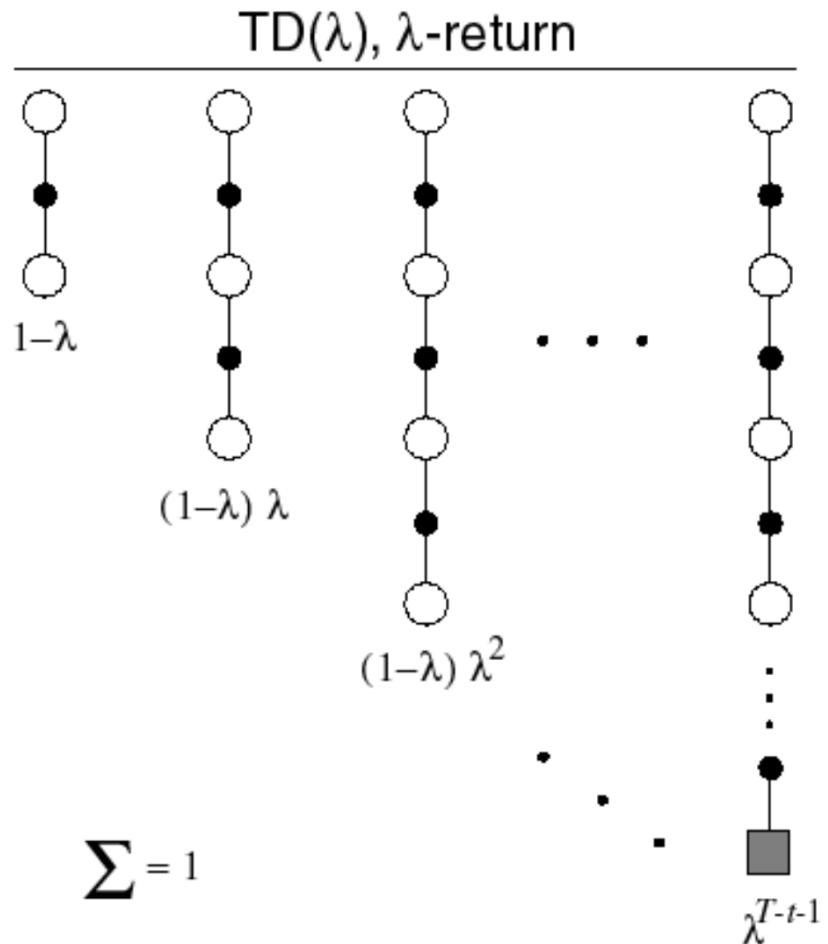
N-Step Return

- N-step temporal-difference learning: Just pick one.

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{(n)} - V(s_t) \right)$$

- The lower the number of steps, the smaller the variance.
- The lower the number of steps, the larger the bias.
- We can pick a (n) to trade off bias with variance.

But why picking? Using them all with λ -return



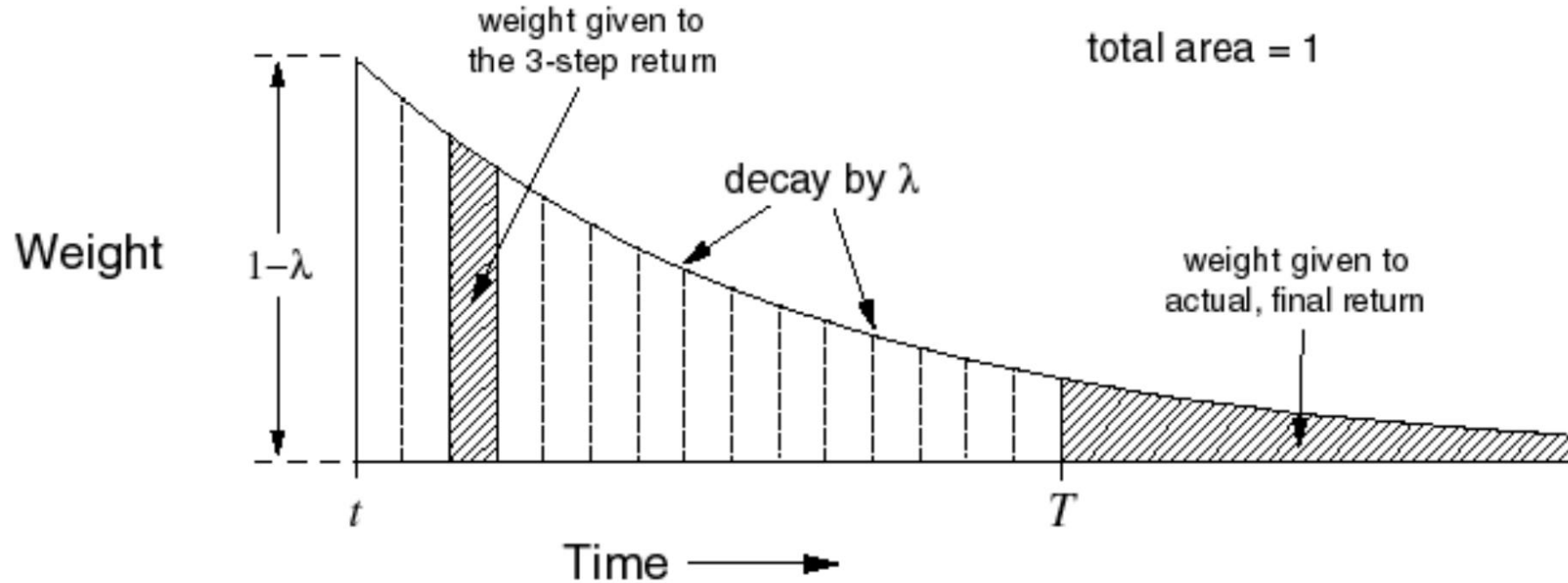
- The λ -return G_t^λ combines *all* n-step returns $G_t^{(n)}$
- Formally: weighted average of $G_t^{(n)}$, using exponentially decaying weight $(1 - \lambda)\lambda^{n-1}$:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- We call this process $TD(\lambda)$

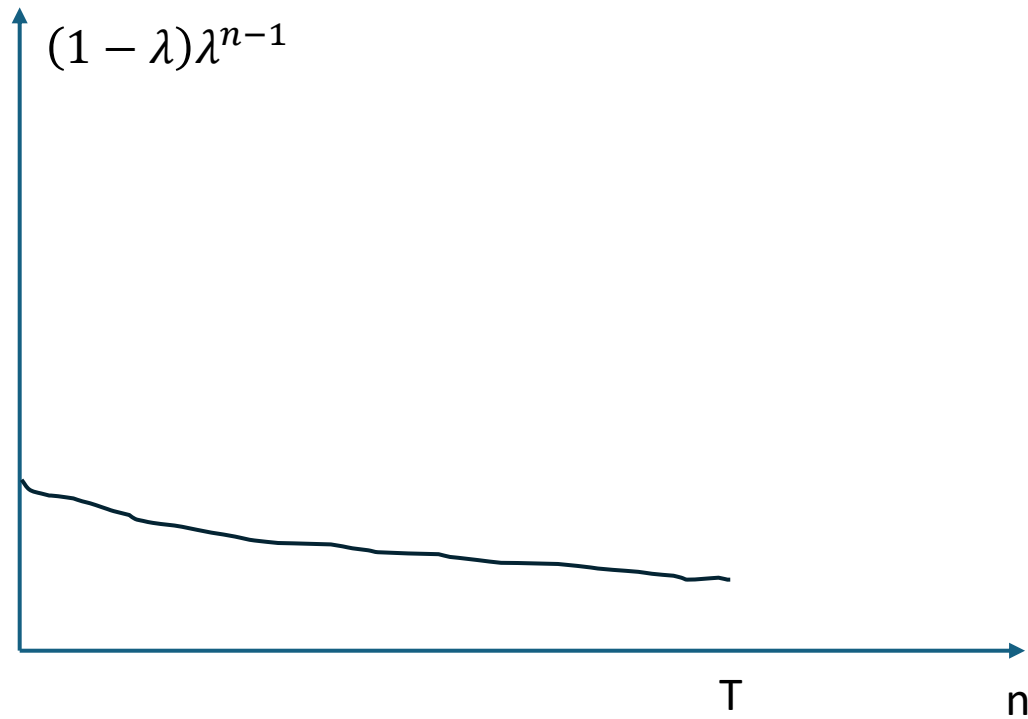
$$V(s_t) \leftarrow V(s_t) + \alpha(G_t^\lambda - V(s_t))$$

$TD(\lambda)$ Weighting Function



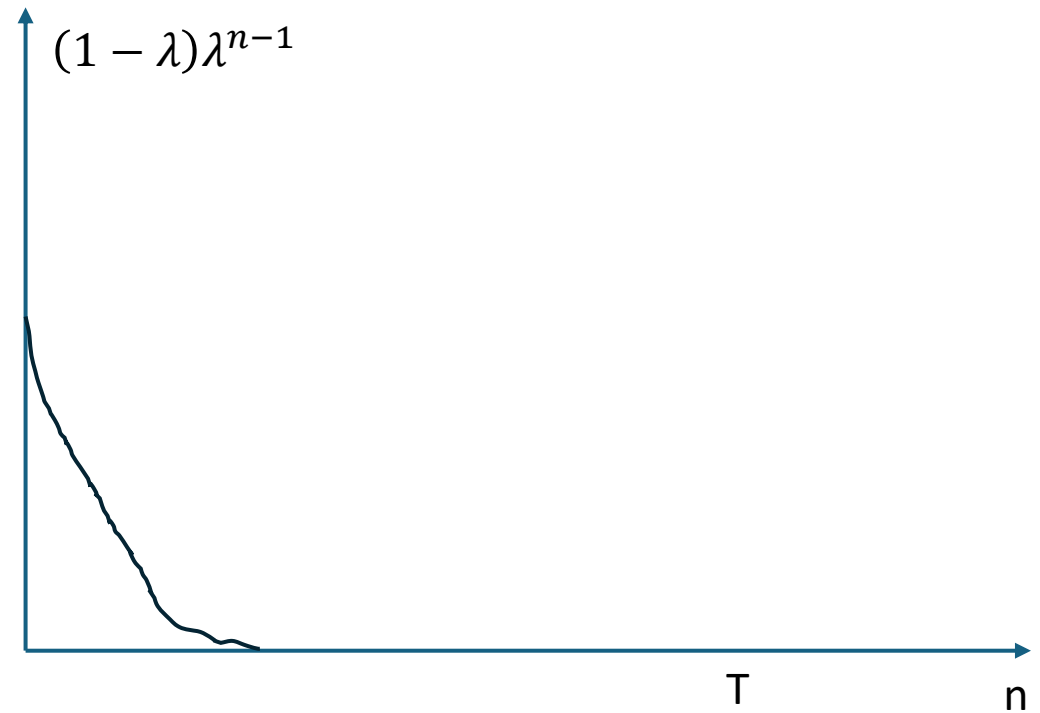
$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

$TD(\lambda)$ Weighting Function



$$\lambda \approx 1$$

Far-sighted 

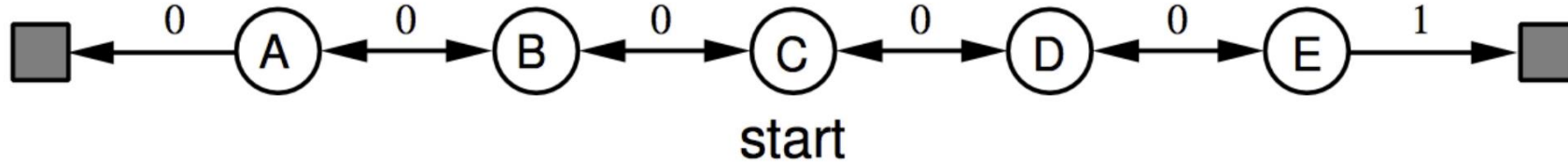


$$\lambda \approx 0$$

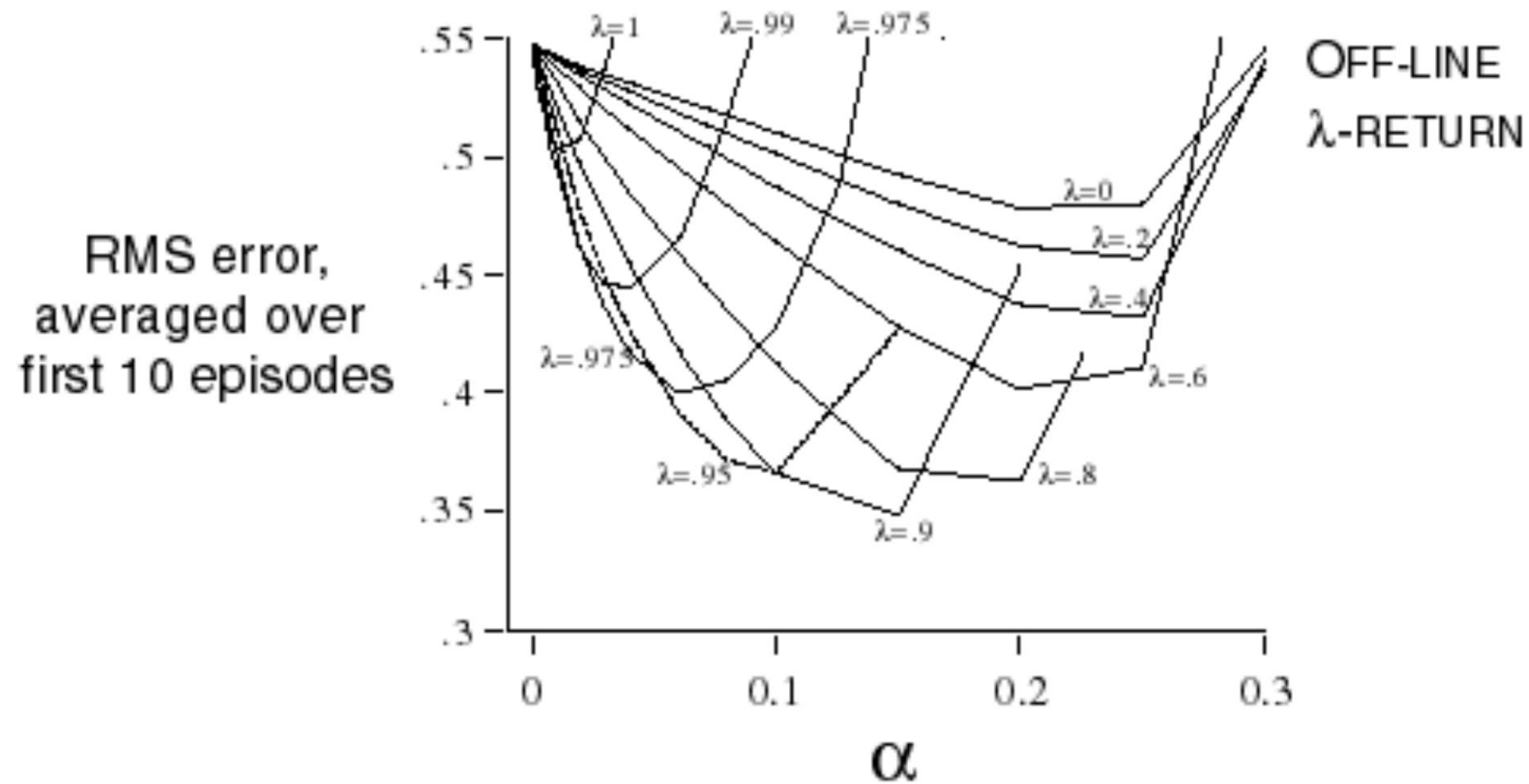
Short-sighted 

Random Walk Example

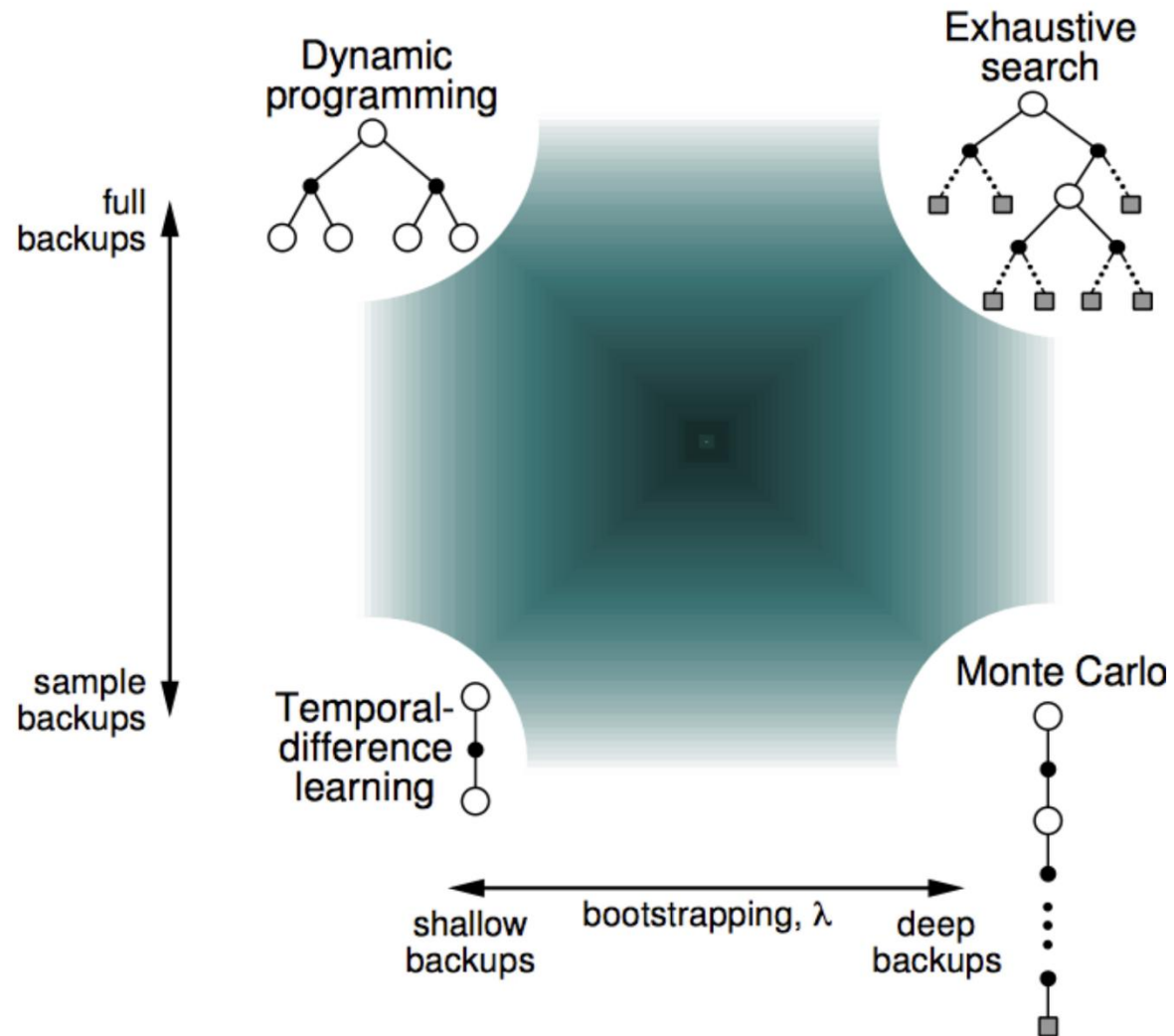
- Find $V(s)$ for the following MRP:



Forward-view of $TD(\lambda)$ on Large Random Walk

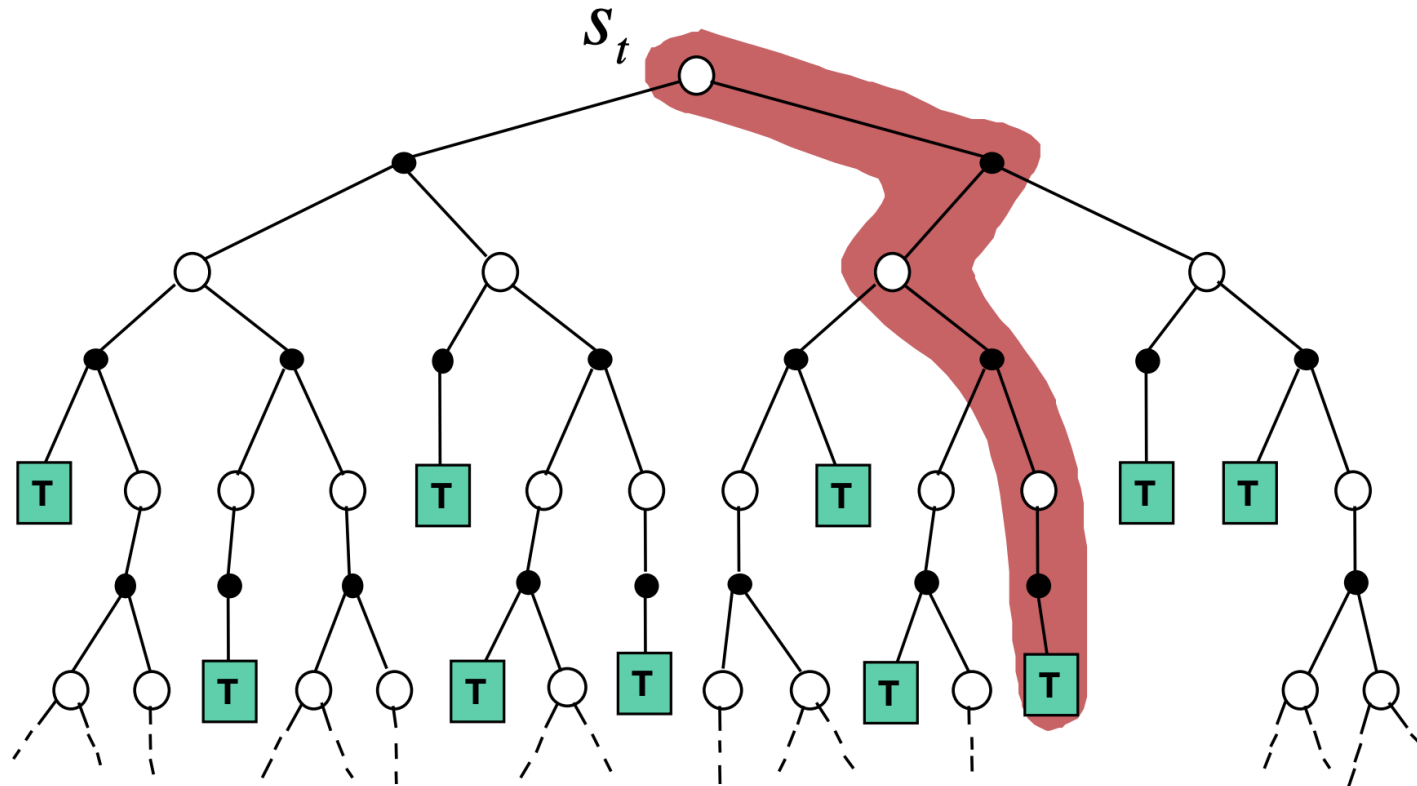


A Unified View of Reinforcement Learning



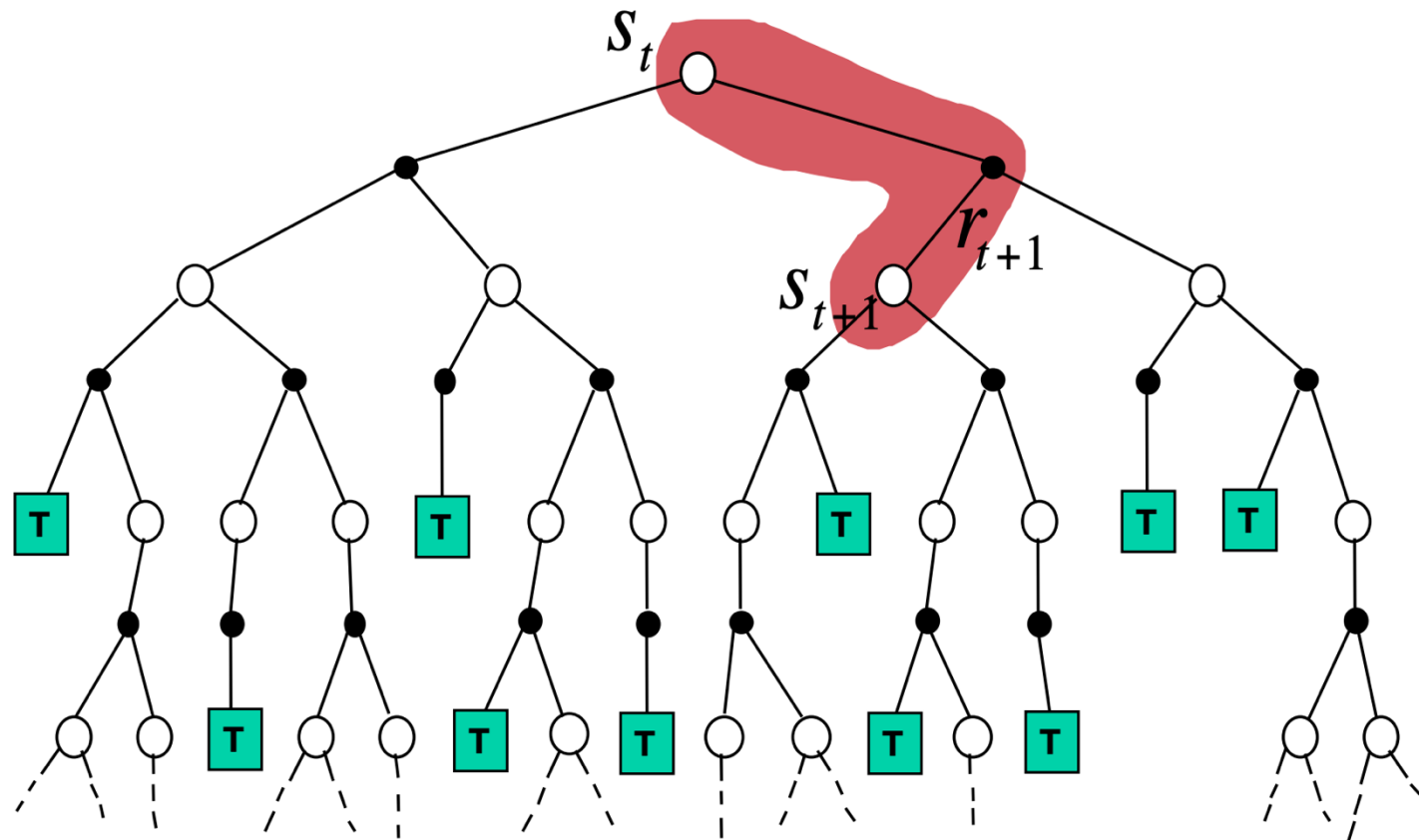
Monte-Carlo Backup

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$



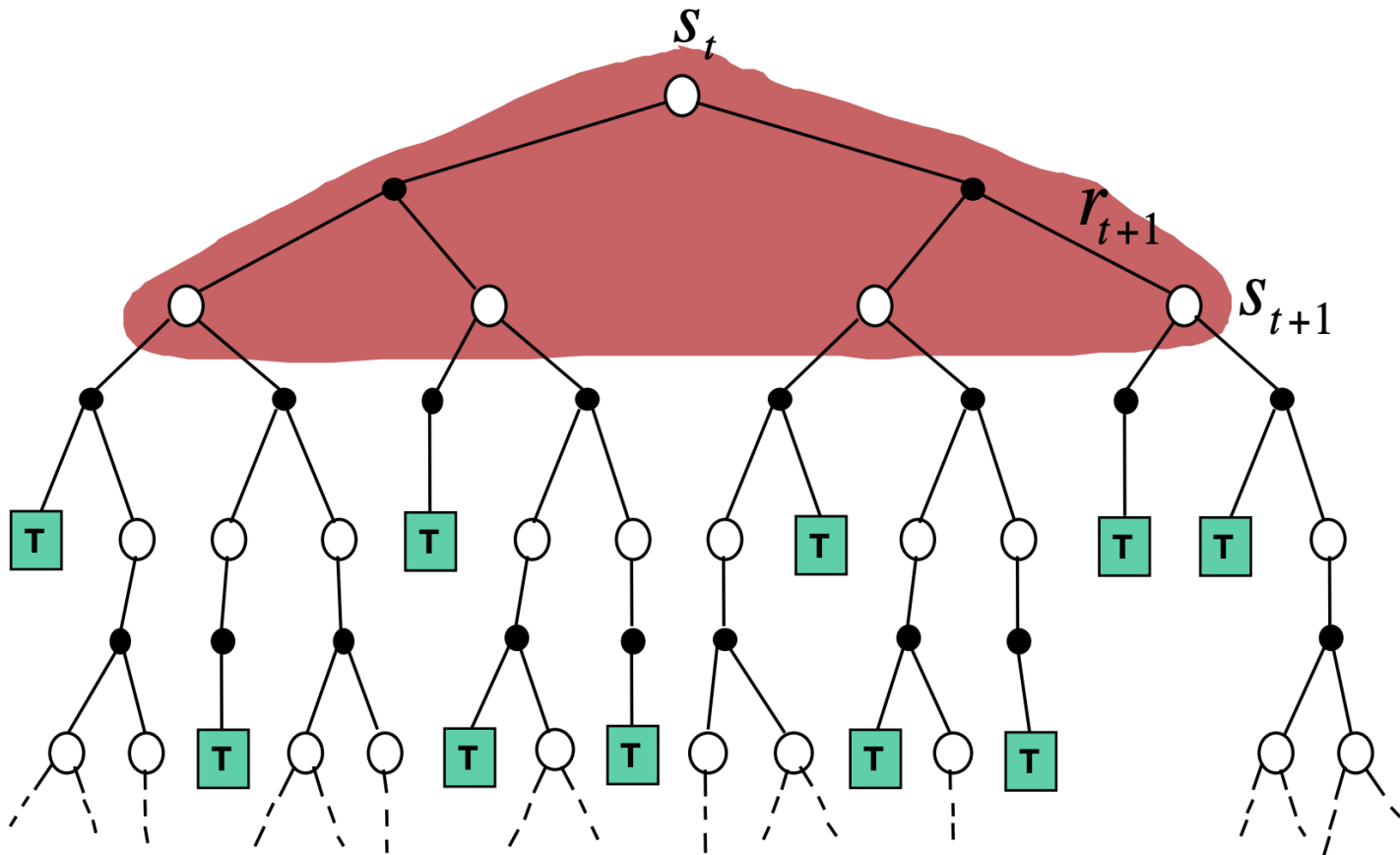
Temporal-Difference Backup

$$V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

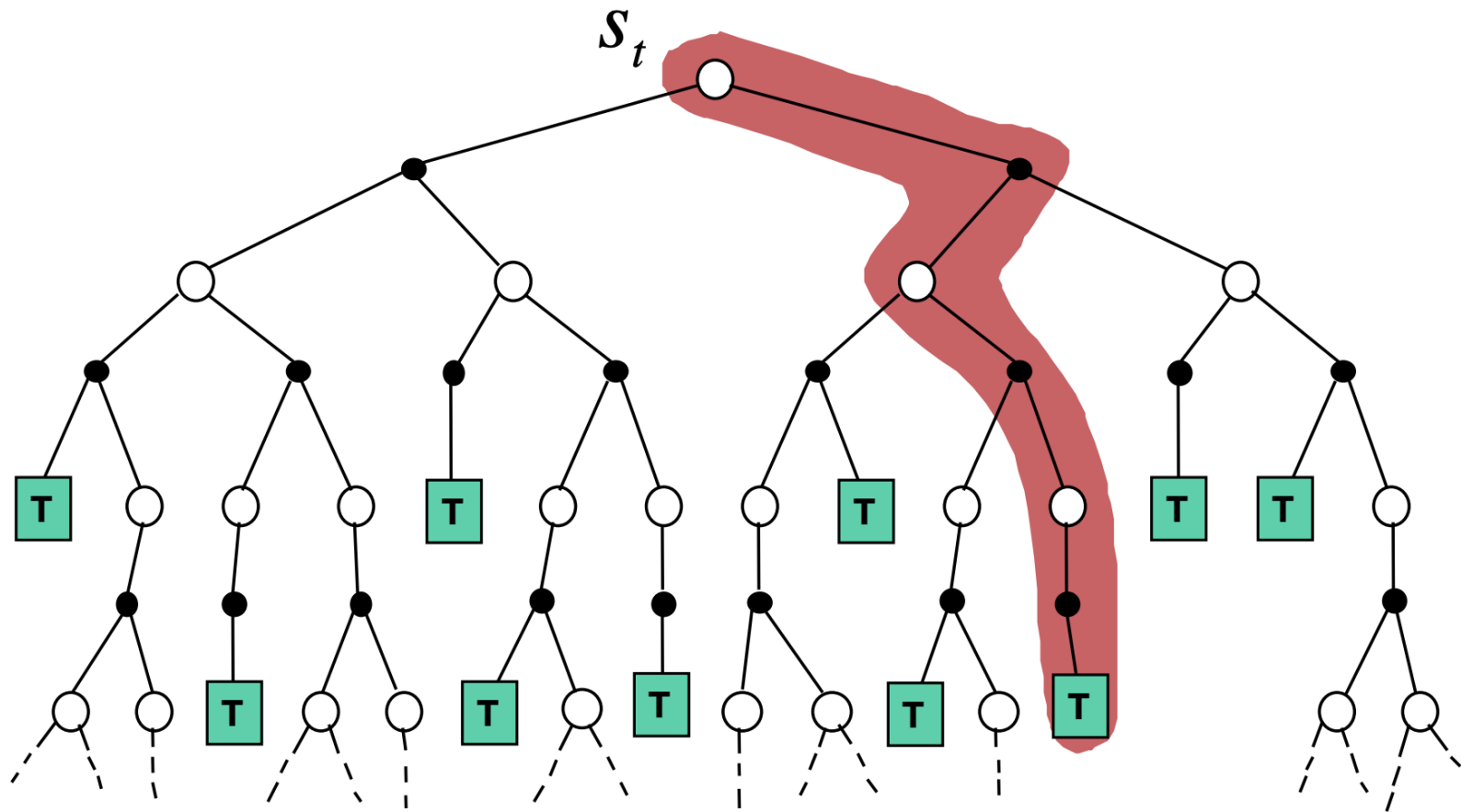


Dynamic Programming Backup

$$V(s_t) \leftarrow \mathbb{E}_{\pi}[r_{t+1} + \gamma V(s_{t+1})]$$



$TD(\lambda)$ Weighting Function



Bootstrapping and Sampling

- **Bootstrapping**: update involves an estimate
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps
- **Sampling**: update samples an expectation
 - MC samples
 - DP does not sample
 - TD samples



Baron Munchausen pulls himself up by his bootstraps!